

编程题

一. 一个整数 n ，使用动态规划，返回和为 n 的完全平方数的最少数量。

(a) 核心算法大致如下：

```
int numSquares(int n) {
    std::vector<int> dp(n + 1, INT_MAX); // 边界条件
    dp[0] = 0;
    //从小到大递推计算每个数字的最优解
    for (int i = 1; i <= n; ++i) {
        //尝试每一个可用的完全平方数 j*j
        for (int j = 1; j * j <= i; ++j) {
            dp[i] = std::min(dp[i], dp[i - j * j] + 1);
        }
    }
    return dp[n];
}
```

定义一个一维动态规划数组 dp ，其中 $dp[i]$ 表示凑出正整数 i 所需的最少完全平方数个数。对于任意正整数 i ，其状态转移方程的思考逻辑为：尝试选择一个不大于 i 的完全平方数 j^2 作为组成 i 的第一个元素。将其减去后，剩余的数为 $i - j^2$ ，此时可以直接调用之前已经计算并存储好的最优解 $dp[i - j^2]$ 。遍历所有可能的 j （满足 $j^2 \leq i$ ），从中挑出能使总积木数最少的方案。

(b) 复杂度分析：

时间复杂度：算法拥有一前一后两层循环。外层循环从 1 遍历到 n ，共执行 n 次；内层循环在每次迭代中尝试所有不大于当前数字 i 的平方数，其最大迭代次数为 $\sqrt{n}$ 。因此，总时间复杂度为 $O(n\sqrt{n})$ 。空间复杂度：算法在执行过程中只需要开辟一个大小为 $n + 1$ 的一维矢量数组 dp 用于记录中间状态，其余仅使用常数级别的辅助变量。因此，总空间复杂度为 $O(n)$ 。

通过编写主函数对不同规模的输入进行测试，控制台输出的测试案例运行结果如下。

```
int main() {
    Solution solution;
    int n = 12;
    cout << "The least number of perfect square numbers that sum to " << n << " is: " << solution.numSquares(n) << endl;

    n = 13;
    cout << "The least number of perfect square numbers that sum to " << n << " is: " << solution.numSquares(n) << endl;

    return 0;
}
```

```
The least number of perfect square numbers that sum to 12 is: 3
The least number of perfect square numbers that sum to 13 is: 2
```

二. 判断是否可以将数组分割成两个子集, 使得两个子集的元素和相等

本题可转化为 0-1 背包问题, 同样采用动态规划方法求解。

要将一个非空数组分割成两个元素和相等的子集, 前提是整个数组的总和 sum 必须为偶数。若 sum 为奇数则直接返回 `false`。若为偶数, 则问题等价于: 从数组中挑选若干个数字, 能否使它们的和恰好等于目标容量 $target = sum / 2$ 。

为了优化空间, 算法采用一维滚动数组 `dp` 进行记录。`dp[j]` 是一个布尔值, 表示是否可以从数组中挑选部分数字刚好凑出累加和为 j 。

核心代码如下:

```
bool canPartition(vector<int>& nums) {
    int sum = accumulate(nums.begin(), nums.end(), 0);
    if (sum % 2 != 0) {
        return false;
    }
    int target = sum / 2;
    vector<bool> dp(target + 1, false);
    dp[0] = true;

    for (int num : nums) {
        for (int j = target; j >= num; --j) {
            dp[j] = dp[j] || dp[j - num];
        }
    }
    return dp[target];
}
```

测试案例如下:

```
int main() {
    Solution solution;
    vector<int> nums = {1, 5, 11, 5};
    bool result = solution.canPartition(nums);
    cout << (result ? "true" : "false") << endl;
    return 0;
}
```

结果如下:

```
PS D:\workspace\algorithm_hw_
true
```

测试案例二为:

```
int main() {  
    Solution solution;  
    vector<int> nums = {1, 2, 3, 5};  
    bool result = solution.canPartition(nums);  
    cout << (result ? "true" : "false") << endl;  
    return 0;  
}
```

结果如下：

```
true  
PS D:\workspace\algorithm_hw  
false
```